FOODVERSE AI

Multimodal Food Intelligence Agent

REQUIREMENT ANALYSIS

Functional, non-functional, technical, and dependency requirements

1. Purpose

This document defines the requirements for Foodverse AI, a multimodal food intelligence application developed as a project prototype. It records the expected user-facing capabilities, the system's quality expectations, the technical direction discussed during development, and the Python dependencies listed for the project. It is limited to Requirement Analysis.

2. Project Overview

Foodverse AI aims to provide food-related assistance through a web interface and backend APIs. Users may provide ingredients and constraints for recipe generation, or submit food-related images where image endpoints are available.

The backend validates requests and returns structured responses for the frontend or API testing tools.

3. Stakeholders and Intended Users

End users: people seeking recipe ideas, food-related image analysis, or meal suggestions.

Project developer/team: responsible for implementation, testing, configuration, and maintenance.

Evaluators/teachers: may review the application, source repository, API behavior, and project documentation.

4. Functional Requirements

FR-1 — Ingredient-based recipe generation

The system should accept an ingredient list and relevant constraints, such as dietary restrictions and maximum preparation time, and return recipe suggestions when the feature is configured. Example input discussed during development: paneer, spinach, cherry tomatoes; vegetarian; maximum preparation time of 30 minutes.

FR-2 — Input validation

The API should validate required fields and handle missing, empty, whitespace-only, or unusual input without an unhandled crash. It should return a clear response indicating success or the validation problem.

FR-3 — Food image input/recognition

Where the food-recognition endpoint is implemented, the system should accept supported image uploads, check that an input file is present and usable, and return a structured result or a meaningful error.

FR-4 — Food freshness-related support

If the freshness feature is enabled, it should return preliminary guidance based on the implemented logic. It must not claim that visual appearance alone proves food is safe to consume.

FR-5 — Kitchen hygiene-related support

If the hygiene feature is enabled, the system should accept the inputs defined by that endpoint and provide structured observations or scoring. Automated detection should only be claimed when implemented and validated.

FR-6 — Structured API responses

API endpoints should return machine-readable responses, normally JSON for standard requests, and suitable status codes for successful requests and errors.

FR-7 — API documentation and testing

During development, the backend should expose interactive API documentation through FastAPI's Swagger UI, typically at the local /docs route, when the server is running.

FR-8 — Frontend integration

The frontend should let users choose an available feature, submit the appropriate text or image input, and view the returned result or error.

5. Non-Functional Requirements

Usability: forms and results should be understandable to users with limited technical knowledge.

Reliability: malformed requests should be handled gracefully, and one feature's failure should not unnecessarily crash the whole backend.

Maintainability: organize endpoints and feature logic in a modular way so features can be tested and updated separately.

Performance: common API requests should return within a reasonable time for the development environment; model-based operations may take longer depending on hardware and configuration.

Compatibility: the web interface should be usable in a modern browser, and the backend should run in the configured Python environment.

Security: API keys and secrets should be stored in environment variables, not committed to GitHub. Uploaded files should be validated.

Transparency: results related to freshness or hygiene must be presented as preliminary assistance, not official safety certification.

6. Technical Requirements

Programming language: Python for the backend.

Backend framework: FastAPI.

Development server: Uvicorn.

API data formats: JSON for structured request/response data and multipart form data for file uploads where needed.

Image processing: Pillow and OpenCV-related utilities, depending on the implemented feature.

AI/model libraries: PyTorch, Torchvision, Ultralytics, and Transformers are included in the dependency list for potential or implemented model workflows.

External service integrations: OpenAI, Groq, and Supabase libraries appear in the dependency list; actual use depends on project configuration and valid credentials.

Version control: Git and GitHub for source code and documentation.

Testing: FastAPI Swagger UI and HTTP requests to validate endpoints.

7. Software and Environment Requirements

A supported Python installation and a project virtual environment (.venv).

A terminal such as Windows PowerShell or Command Prompt to install dependencies and start the backend.

Required packages installed from the project's requirements.txt file.

Environment variables and valid API credentials for any external services that are actually used.

For deployment, a hosting environment configured with the correct start command, dependencies, and environment variables.

8. Python Dependencies — requirements.txt

The package names below are the dependency list recovered from our earlier Foodverse AI project discussion. The previously recorded list did not include pinned version numbers, so the entries are shown without versions. Please compare this appendix with the current requirements.txt in your project folder before treating it as an exact byte-for-byte copy.

fastapi uvicorn ultralytics transformers torch torchvision opencv-python-headless Pillow numpy pandas scikit-learn openai groq SpeechRecognition pydub python-multipart python-dotenv pytesseract supabase requests fpdf

9. Constraints and Assumptions

This project is being developed as a prototype, and the level of completion may differ between features.

A package being present in requirements.txt does not prove that its related feature is fully implemented or currently used.

Some AI or model workflows may require large downloads, suitable hardware, network access, or external API keys.

Freshness and hygiene outputs are advisory and cannot replace professional inspection or food-safety procedures.

Deployment may require separate configuration for frontend and backend services, including API base URLs and cross-origin access settings.

10. Acceptance Criteria

The backend starts successfully in the intended environment.

Implemented endpoints are documented and can be tested through /docs or an HTTP client.

Valid input returns a structured response; invalid input returns a controlled error.

Recipe generation accepts the supported ingredient and constraint fields.

Implemented image endpoints handle supported uploads and report errors clearly.

Secrets are configured outside source code.

Documentation accurately distinguishes working features from planned or partially implemented features.